Model-Based Systems Engineering

Relazione di Progetto

Generazione di Implementazione Java da Modello UML

Caso di Studio: *ATC System*

Componenti del gruppo

Antonio D'Alessandro	**Matricola:** 0382940
Mihai Alexandru Sandu	**Matricola:** 0378481
Matteo Sabelli	**Matricola:** 0382089

Indice

1 Introduzione **1**

1.1 Contesto e motivazione 1

1.2 Struttura della relazione 1

2 Requisiti della Task e Tracciabilità **2**

2.1 Requisiti funzionali 2

2.2 Tabella di tracciabilità Requisiti 2

3 Ambiente di sviluppo e Struttura del Workspace **4**

3.1 Strumenti utilizzati 4

3.2 Progetti del workspace 4

4 Profilo UML e Stereotipi **6**

4.1 Motivazione dell'uso del profilo 6

4.2 Stereotipi del profilo ATC 6

4.3 Applicazione nel modello ATC 6

5 Modello UML del Sistema ATC **8**

5.1 Vista architetturale: Package Diagram 8

5.2 Class Diagram: core 10

5.2.1 Panoramica 10

5.2.2 Classi e responsabilità 10

5.2.3 Associazioni principali 11

5.2.4 Operazioni principali 11

5.3 Class Diagram: domain 13

5.3.1 Panoramica 13

5.3.2 Classi e responsabilità 13

5.3.3 Ereditarietà 14

5.3.4 Associazioni principali 14

5.3.5 Operazioni principali 15

5.3.6 Relazioni con gli altri package 15

5.4 Class Diagram: service 16
5.4.1 Panoramica 16
5.4.2 Interfacce 16
5.4.3 Classi 17
5.4.4 Enumerazioni 18
5.4.5 Associazioni principali 18
5.4.6 Gestione dei conflitti 19
5.4.7 Nota sugli aspetti periodici 19
5.5 Class Diagram: comms 20
5.5.1 Panoramica 20
5.5.2 Interfacce 20
5.5.3 Classi 21
5.5.4 Enumerazioni 22
5.5.5 Associazioni principali 22
5.5.6 Implicazioni sulla generazione Java 23

6 Trasformazione Acceleo 24
6.1 Introduzione alla generazione Acceleo 24
6.2 Struttura del modulo `generate.mtl` 24
6.2.1 Intestazione del modulo e template di ingresso 25
6.2.2 Guard di template, cosa generare 25
6.2.3 Direttiva `package` e area protetta degli import 26
6.2.4 Firma della classe: `extends` e `implements` 26
6.2.5 Generazione attributi: proprietà e associazioni navigabili 26
6.2.6 Costruttori, getter/setter 27
6.2.7 Operazioni: owned e metodi richiesti da interfacce/astratti . . . 27
6.2.8 Aree protette nei metodi 27
6.3 Gestione degli stereotipi del profilo ATC 27
6.4 Gestione delle associazioni: casi e decisioni 28
6.4.1 Molteplicità e tipi: riferimento singolo vs collezione 28
6.4.2 Caso 1: associazione 1–1 (riferimento obbligatorio) 29
6.4.3 Caso 2: associazione 0–1 (riferimento opzionale) 29
6.4.4 Caso 3: associazione 0–* / 1–* (collezione) 29
6.4.5 Composizione vs associazione 29
6.4.6 Associazioni presenti nel modello ma non generate come campi . 30
6.4.7 Bidirezionalità: scelta progettuale 30
6.5 Gestione dell'ereditarietà: casi e decisioni 30
6.5.1 Ereditarietà tra classi: `extends` 30
6.5.2 Interfacce: `implements` e generazione degli stub 31

6.5.3 Metodi da implementare 31

7 Parsing **33**

7.1 Introduzione 33

7.2 Entry point: `Main.java` 33

7.3 Componenti principali 36

7.3.1 `PapyrusUmlSaxParser` 36

7.3.2 `UmlModel` 36

7.3.3 `UmlType` 36

7.3.4 `JavaCodeGenerator` 37

7.4 Risultato dell'approccio e funzionamento complessivo 37

8 Confronto tra i due approcci **38**

8.1 Introduzione 38

8.2 Approccio Acceleo: punti di forza e criticità 38

8.2.1 Pro 38

8.2.2 Contro 39

8.3 Approccio Java + SAX: punti di forza e criticità 39

8.3.1 Pro 39

8.3.2 Contro 40

9 Conclusioni **41**

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Il caso di studio è un sistema di Air Traffic Control (ATC) modellato in quattro package principali (core, domain, service, comms) in accordo ai requisiti REQ1–REQ5. A partire dal modello UML abbiamo realizzato e valutato due approcci di generazione automatica del codice: (i) una trasformazione Model-to-Text con Eclipse Acceleo (MTL), e (ii) una generazione general-purpose sviluppata in Java, basata sul parsing del file UML/XMI tramite la libreria SAX. In entrambi i casi il generatore produce classi Java coerenti con il modello, con package corretti, attributi, metodi e costruttori. Infine, discutiamo scelte di progetto, tracciabilità rispetto ai requisiti e un set di verifiche di coerenza modello–codice, mettendo a confronto pro e contro dei due approcci in termini di semplicità, manutenibilità, estendibilità e controllo fine sul processo di generazione. Il codice prodotto deve essere coerente con il modello e considerare:

- relazioni di **associazione** (incluse molteplicità e navigabilità);
- relazioni di **ereditarietà** (generalizzazioni e implementazioni di interfacce);
- **attributi** e **metodi** delle classi (owned attributes e owned operations).

1.2 Struttura della relazione

Il documento presenta requisiti e tracciabilità, descrive il modello e il profilo UML, definisce le regole UML→Java, illustra l'implementazione Acceleo e valuta l'output Java generato.

Capitolo 2

Requisiti della Task e Tracciabilità

2.1 Requisiti funzionali

I requisiti funzionali descrivono in modo chiaro e verificabile quali servizi e comportamenti il sistema deve offrire, indipendentemente da come verranno implementati. In altre parole, definiscono cosa il sistema deve fare (es. gestire aeromobili, acquisire tracce radar, rilevare conflitti, gestire comunicazioni), specificando le funzionalità attese e i dati coinvolti. Nel seguito, tali requisiti vengono messi in relazione con gli elementi del modello UML tramite una tabella di tracciabilità, così da evidenziare in modo diretto dove e come ciascun requisito è rappresentato nel modello.

2.2 Tabella di tracciabilità Requisiti

Req.	**Descrizione sintetica**	**Elementi UML coinvolti**
REQ1	Gestione aeromobili (posizione, quota, velocità)	`domain.Aircraft`, `domain.Position`, attributi *altitude/speed/position*, metodi di update
REQ2	Radar periodico e tracciamenti	`service.TrackingComponent`, stereotipo *Periodic*
REQ3	Tracking e aggiornamento registry	`service.TrackingComponent`, `core.AircraftRegistry`, `core.ATCSystem`
REQ4	Conflict detection	`service.IConflictDetector`, `service.BasicConflictDetector`, `service.Conflict`, `service.ConflictType`

REQ5	Comunicazione VHF	`comms.CommunicationManager`, `comms.CommunicationSession`, `comms.VHFChannel`, `comms.Message`

Capitolo 3

Ambiente di sviluppo e Struttura del Workspace

3.1 Strumenti utilizzati

gli strumenti utilizzati per svolgere il progetto sono:

- **Eclipse IDE**: ambiente di sviluppo integrato utilizzato come piattaforma del progetto; consente di gestire workspace e progetti, installare plugin di modellazione e avviare l'esecuzione della trasformazione Acceleo.
- **Papyrus / UML2**: Papyrus è lo strumento di modellazione UML integrato in Eclipse; si basa su **Eclipse UML2**, implementazione EMF dello standard UML, per creare e manipolare modelli UML (diagrammi e metadati).
- **EMF (Eclipse Modeling Framework)**: framework per la definizione di metamodeli e la gestione dei modelli (API di navigazione, serializzazione, strutture dati). Nel progetto costituisce la base tecnologica che permette ad Acceleo di leggere il modello UML e iterare su classi, attributi, associazioni e operazioni.
- **Acceleo**: strumento *Model-to-Text (M2T)* che, a partire dal modello UML (tramite EMF/UML2), esegue template `.mtl` per generare automaticamente codice sorgente (in questo caso Java).
- **JDK (Java Development Kit)**: kit di sviluppo Java necessario per compilare ed eventualmente eseguire il codice generato; include compilatore (`javac`), librerie standard e strumenti di runtime.

3.2 Progetti del workspace

Il workspace include:

- **ATC_Model**: modello UML (file `.uml`, `.di`, `.notation`);

- **ATC_Profile**: profilo UML (stereotipi *Periodic* e *Subsystem*);
- **org.eclipse.acceleo.module.sample**: modulo Acceleo `generate.mtl` e output generato.
- **atc-java-generator**: progetto Maven per la generazione general purpose

Capitolo 4

Profilo UML e Stereotipi

4.1 Motivazione dell'uso del profilo

Nel progetto è stato introdotto un profilo UML per estendere il modello con informazioni specifiche del dominio ATC senza alterare lo standard UML. Tramite stereotipi e valori etichettati è possibile aggiungere metadati (es. aspetti temporali o di criticità) che guidano sia l'analisi sia, quando necessario, la trasformazione M2T. In questo modo il modello rimane focalizzato sul *cosa* (struttura e responsabilità delle classi), mentre il profilo concentra *metadati* e regole aggiuntive, migliorando manutenibilità ed estensibilità.

4.2 Stereotipi del profilo ATC

Il profilo `ATC_Profile` definisce tre stereotipi principali:

- ***Periodic*** (su `Class`): caratterizza componenti a comportamento periodico tramite l'attributo `periodMs`.
- ***SafetyCritical*** (su `Class`): identifica componenti rilevanti per la sicurezza, con livello di criticità `CriticalityLevel` (es. `LOW`, `MEDIUM`, `HIGH`).
- ***Subsystem*** (su `Package`): consente di etichettare package come sottosistemi architetturali.

4.3 Applicazione nel modello ATC

Nel modello `ATC_Model` gli stereotipi sono applicati principalmente a classi dei package `core`, `service` e `comms`.

Applicazioni di *Periodic*. Lo stereotipo *Periodic* è applicato a `service.TrackingComponent` con `periodMs = 1000`, a indicare un componente che opera con cadenza regolare (1 secondo) per attività come polling/aggiornamento.

Applicazioni di SafetyCritical. Lo stereotipo *SafetyCritical* è applicato a più classi per evidenziare componenti sensibili:

- `core.ATCSystem` e `core.AircraftRegistry` con `criticality = MEDIUM`;
- `domain.Aircraft`, `service.RadarTrack` e `comms.VHFChannel` con `criticality = LOW`;
- `service.TrackingComponent`, `service.BasicConflictDetector` e `comms.CommunicationManager` con `criticality = HIGH`.

Applicazioni di *Subsystem*. Lo stereotipo *Subsystem* è definito nel profilo per categorizzare package/sottosistemi ed è stato applicato al package `service` e `comms`.

Come vengono sfruttati nella generazione. Nella versione attuale del generatore Acceleo, gli stereotipi *Periodic* e *SafetyCritical* vengono utilizzati direttamente nella trasformazione. In particolare, se una classe è marcata *Periodic* e `periodMs = 1000`, vengono generati un attributo `periodMs` e la sua inizializzazione nel costruttore. Inoltre, se una classe è marcata *SafetyCritical*, viene generato l'attributo `criticality`, valorizzato in base al tagged value omologo, che può assumere i tre livelli `LOW`, `MEDIUM` o `HIGH`. Lo stereotipo *Subsystem* è invece mantenuto come metadato di analisi/documentazione e non viene tradotto in costrutti Java.

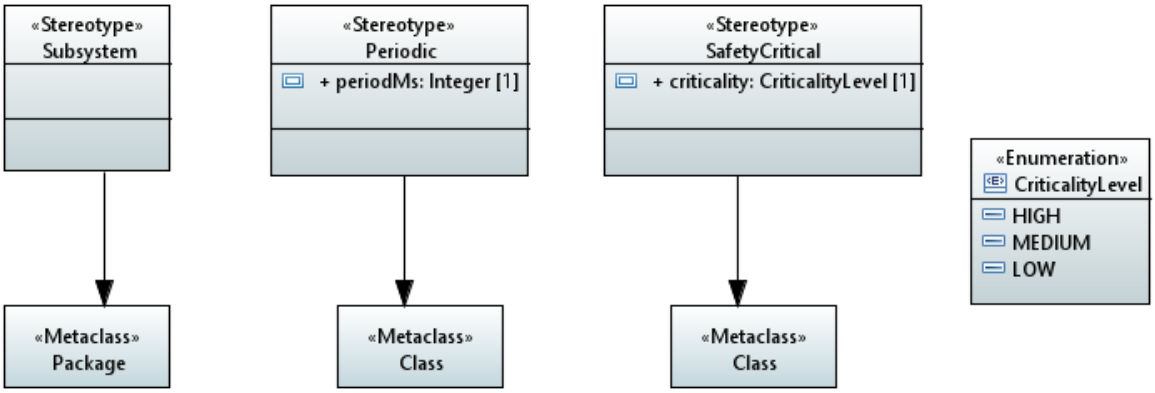

Figura 4.1: Profilo UML `ATC_Profile` e stereotipi definiti.

Capitolo 5

Modello UML del Sistema ATC

5.1 Vista architetturale: Package Diagram

Il modello UML del sistema ATC è organizzato in quattro package principali, ciascuno con responsabilità ben definita. Tale suddivisione migliora la leggibilità del modello e favorisce una generazione del codice Java coerente con l'architettura logica del sistema.

`core` Raccoglie i componenti di **coordinamento** e le strutture centrali del sistema. In particolare include `ATCSystem`, che rappresenta il punto di orchestrazione del comportamento complessivo, e `AircraftRegistry`, responsabile della **gestione e consultazione** dell'insieme di aeromobili noti al sistema.

`domain` Contiene le **entità principali del dominio** e i relativi dati: ad esempio `Aircraft` e `Position` per rappresentare lo stato di un aeromobile (posizione, quota, velocità), e la gerarchia `Person`–`Operator`/`Commander` per modellare gli attori coinvolti. Questo package rappresenta il nucleo informativo su cui operano i servizi.

`service` Include i **servizi applicativi** legati a radar, tracking e rilevazione dei conflitti. Comprende le interfacce `IRadarSurveillance` e `IConflictDetector`, le classi `RadarTrack` e `TrackingComponent` per l'acquisizione/gestione delle tracce, e le classi `BasicConflictDetector` e `Conflict` (con l'enumerazione `ConflictType`) per la **conflict detection**. In questo package ricadono anche gli aspetti periodici.

`comms` Modella il sottosistema di **comunicazione** e i canali VHF. Include l'interfaccia `ICommChannel`, l'implementazione `VHFChannel`, la gestione delle sessioni tramite `CommunicationManager` e `CommunicationSession`, e lo scambio di informazioni attraverso `Message`. Le enumerazioni

`ChannelStatus` e `SessionStatus` rappresentano gli stati di canale e sessione.

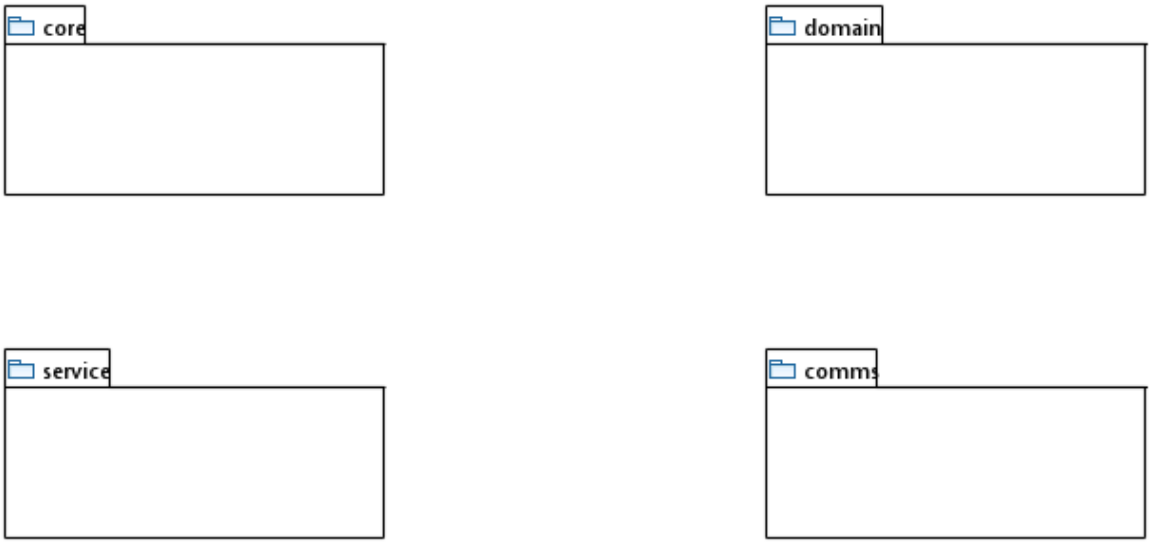

Figura 5.1: Package Diagram

5.2 Class Diagram: core

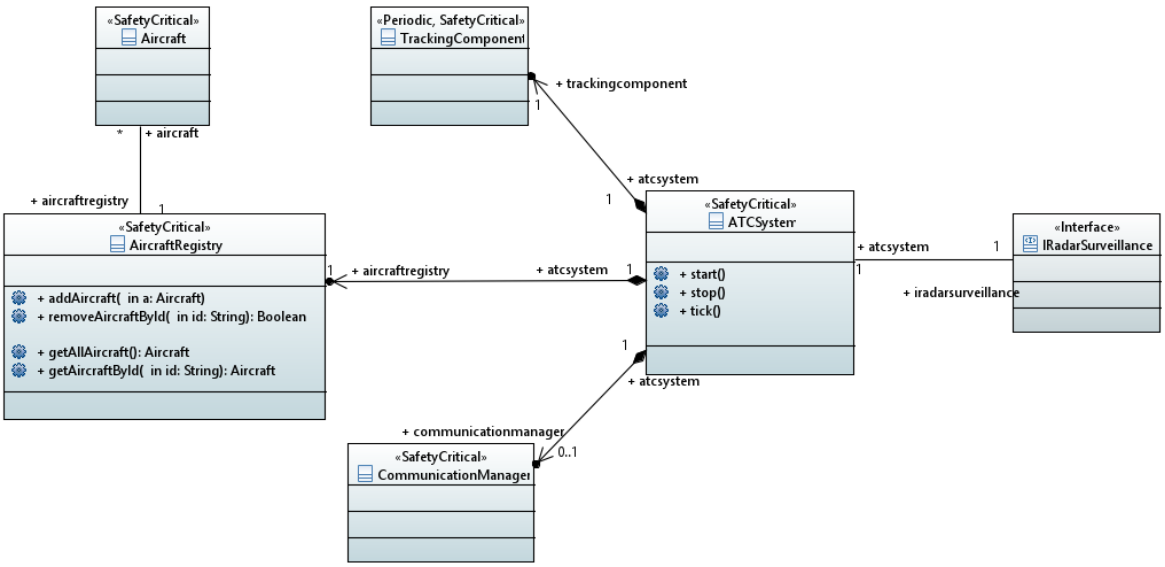

Figura 5.2: Cr_ClassDiagram

5.2.1 Panoramica

Il package `core` raccoglie i componenti di **coordinamento** del sistema ATC e le strutture centrali per la gestione dello stato globale. In particolare, esso include il *controller* principale (`ATCSystem`) e il registro degli aeromobili (`AircraftRegistry`), che rappresentano il punto di integrazione tra dominio (`domain`), servizi applicativi (`service`) e comunicazioni (`comms`).

5.2.2 Classi e responsabilità

`ATCSystem`. Rappresenta il componente di **orchestrazione** del sistema: coordina l'avvio e l'arresto del sistema e scandisce l'evoluzione del comportamento tramite un ciclo logico (tick). Dal punto di vista strutturale, `ATCSystem` **compone** i principali sottocomponenti necessari al funzionamento:

- un **registro** (`AircraftRegistry`) per memorizzare e consultare gli aeromobili gestiti;
- un **componente di tracking** (`TrackingComponent`) per acquisire aggiornamenti e produrre informazioni operative (ad esempio conflitti);
- un **gestore comunicazioni** (`CommunicationManager`) per la gestione dei canali e delle sessioni VHF.

`AircraftRegistry`. Modella il repository degli aeromobili noti al sistema. Il diagramma lo collega concettualmente alle istanze di `domain.Aircraft` con molteplicità `0..*`, indicando che il registro gestisce un insieme (potenzialmente vuoto) di aeromobili. Le operazioni esposte permettono di inserire, rimuovere e recuperare aeromobili, sia singolarmente (per identificativo) sia come collezione.

5.2.3 Associazioni principali

Nel diagramma `core` sono rilevanti le seguenti relazioni:

- **Composizione** `ATCSystem` ◇–`AircraftRegistry` con molteplicità `1..1` sul lato `AircraftRegistry`: il registro è parte integrante del sistema e ne segue il ciclo di vita.
- **Composizione** `ATCSystem` ◇–`TrackingComponent` con molteplicità `1..1` sul lato `TrackingComponent`: il sistema include sempre un componente di tracking.
- **Composizione opzionale** `ATCSystem` ◇–`CommunicationManager` con molteplicità `0..1` sul lato `CommunicationManager`: il sottosistema di comunicazione può essere presente oppure omesso a seconda dello scenario.
- **Associazione** `AircraftRegistry` – `domain.Aircraft` con molteplicità `0..*` sul lato `Aircraft`: il registro gestisce un insieme di aeromobili.
- **Associazione concettuale** `ATCSystem` – `service.IRadarSurveillance`: la presenza di tale relazione evidenzia che, a livello di sistema, è disponibile un servizio radar (in pratica utilizzato tramite i componenti del package `service`).

5.2.4 Operazioni principali

Operazione	Descrizione
`void start()`	Avvia il sistema e inizializza i sottocomponenti necessari al funzionamento.
`void stop()`	Arresta il sistema e gestisce la chiusura/terminazione dei sottocomponenti.
`void tick()`	Esegue un passo del ciclo logico del sistema (es. aggiornamenti periodici e coordinamento dei servizi).

Operazioni di `ATCSystem`.

Operazione	Descrizione
`void addAircraft(Aircraft a)`	Inserisce un aeromobile nel registro.
`Boolean removeAircraftById(String id)`	Rimuove un aeromobile dato l'identificativo; ritorna `true/false` in base all'esito.
`List<Aircraft> getAllAircraft()`	Restituisce l'insieme degli aeromobili gestiti (molteplicità UML `0..*`).
`Aircraft getAircraftById(String id)`	Restituisce l'aeromobile con dato `id` se presente (tipicamente `0..1`).

Operazioni di `AircraftRegistry`.

5.3 Class Diagram: domain

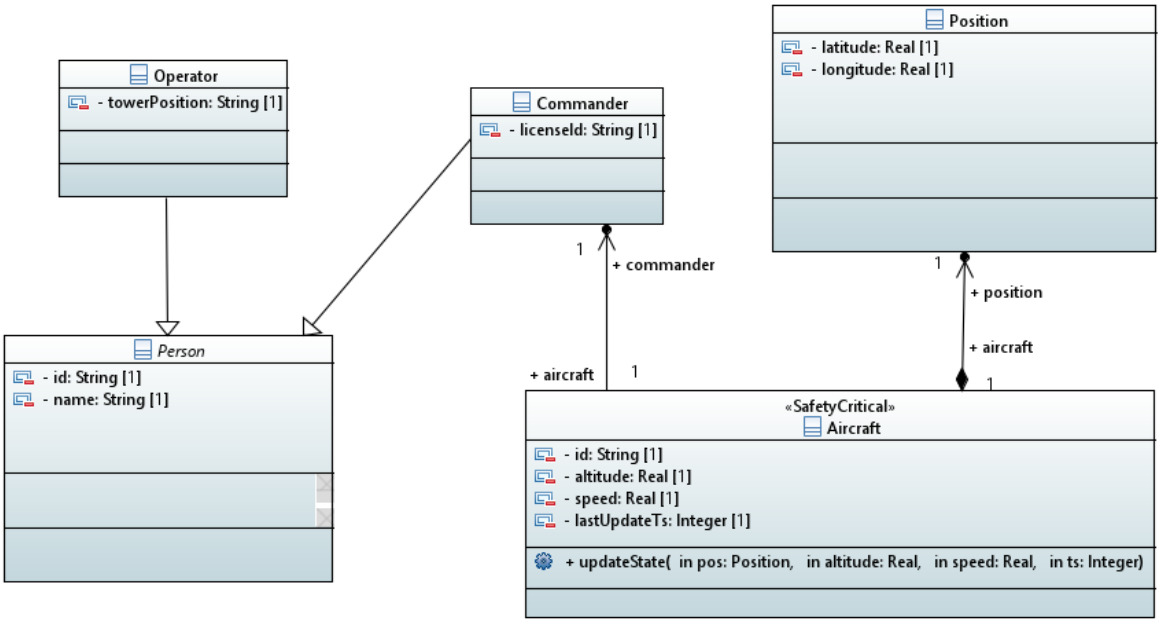

Figura 5.3: Dm_ClassDiagram

5.3.1 Panoramica

Il package `domain` contiene le entità fondamentali del dominio ATC e i relativi dati: aeromobili, posizioni e attori coinvolti (operatore e comandante). Queste classi costituiscono la base informativa su cui operano i servizi (tracking, conflict detection) e il sottosistema di comunicazione.

5.3.2 Classi e responsabilità

`Position`. Rappresenta una posizione geografica elementare, definita da:

- `latitude : Real` (latitudine);
- `longitude : Real` (longitudine).

La classe è utilizzata sia per descrivere la posizione di un `Aircraft` sia all'interno delle tracce radar (`service.RadarTrack`).

`Person` **(astratta).** `Person` è definita nel modello UML come classe astratta (`isAbstract=true`) e fornisce gli attributi comuni agli attori:

- `id : String`;
- `name : String`.

Essa viene specializzata in ruoli più specifici utilizzati nelle comunicazioni e nelle operazioni del sistema.

`Operator.` Specializzazione di `Person` che modella l'operatore (ad es. operatore di torre/controllo). Introduce:

- `towerPosition : String` (posizione/identificativo della postazione operativa).

`Commander.` Specializzazione di `Person` che modella il comandante/pilota associato ad un aeromobile. Introduce:

- `licenseId : String` (identificativo licenza).

`Aircraft.` Rappresenta l'entità principale del dominio: un aeromobile gestito dal sistema ATC. Include attributi per identificazione e stato:

- `id : String` identificativo dell'aeromobile;
- `altitude : Real` quota;
- `speed : Real` velocità;
- `lastUpdateTs : Integer` timestamp dell'ultimo aggiornamento;

e riferimenti strutturali verso:

- `position : Position` (posizione corrente);
- `commander : Commander` (comandante associato).

5.3.3 Ereditarietà

Il diagramma prevede una gerarchia semplice e mirata:

- `Operator` **estende** `Person`;
- `Commander` **estende** `Person`;
- `Person` è **astratta** nel modello, fungendo da base comune per gli attori.

Questa scelta evita duplicazioni (es. `id`, `name`) e rende naturale l'uso degli attori nelle operazioni del package `comms` (sessioni tra operatore e comandante).

5.3.4 Associazioni principali

Nel package `domain` emergono due relazioni strutturali centrali:

- **Composizione** `Aircraft` – `Position`: `Aircraft` possiede l'attributo `position` con molteplicità `1..1` ed è marcato come **composite** nel modello. Ciò indica che la posizione fa parte dello stato interno dell'aeromobile e ne segue il ciclo di vita.

- **Associazione** `Aircraft` – `Commander`: `Aircraft` possiede l'attributo `commander` con molteplicità `1..1`, imponendo che ogni aeromobile sia associato esattamente a un comandante.

5.3.5 Operazioni principali

L'unica operazione owned nel diagramma `domain` è definita su `Aircraft`:

Operazione	Descrizione
`void updateState(Position pos, Real altitude, Real speed, Integer ts)`	Aggiorna lo stato dell'aeromobile in base a una nuova misura (posizione, quota, velocità) e registra il timestamp dell'aggiornamento.

Tabella 5.1: Operazione principale di `Aircraft` nel package `domain`.

5.3.6 Relazioni con gli altri package

Le entità di `domain` sono utilizzate trasversalmente:

- `service.TrackingComponent` aggiorna gli oggetti `Aircraft` (tramite `updateState`) utilizzando dati provenienti da `service.RadarTrack`;
- `comms.CommunicationSession` collega `Operator` e `Commander` per modellare sessioni di comunicazione operative.

5.4 Class Diagram: service

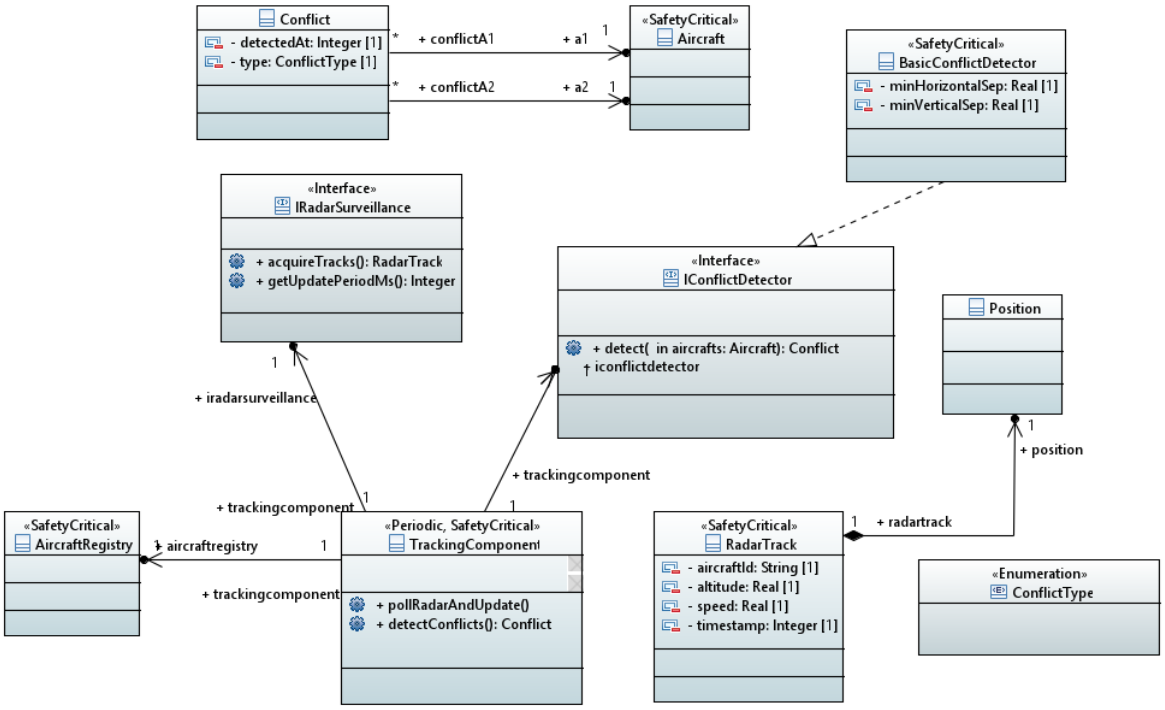

Figura 5.4: Sv_ClassDiagram

5.4.1 Panoramica

Il package `service` contiene i **servizi applicativi** del sistema ATC legati a: (i) acquisizione delle tracce radar, (ii) aggiornamento periodico dello stato degli aeromobili, (iii) rilevazione dei conflitti e loro classificazione. Il diagramma integra elementi del dominio (`domain.Aircraft`, `domain.Position`) e del core (`core.AircraftRegistry`), evidenziando la natura *cross-package* dei servizi.

5.4.2 Interfacce

`IRadarSurveillance`. Modella un servizio radar astratto che fornisce tracce (`RadarTrack`) e un periodo di aggiornamento.

Operazione	Descrizione
`RadarTrack[*] acquireTracks()`	Acquisisce l'insieme delle tracce radar correnti. La molteplicità `[*]` indica un insieme (potenzialmente vuoto) di tracce.
`Integer getUpdatePeriodMs()`	Restituisce il periodo (in millisecondi) con cui il radar aggiorna le tracce.

Tabella 5.2: Operazioni di `IRadarSurveillance`.

`IConflictDetector.` Rappresenta l'algoritmo di **conflict detection** in forma astratta, separando l'interfaccia dall'implementazione concreta.

Operazione	Descrizione
`Conflict[*] detect(Aircraft[*] aircrafts)`	Calcola i conflitti a partire da un insieme di aeromobili. L'input `Aircraft[*]` e l'output `Conflict[*]` esprimono rispettivamente collezioni di aeromobili e conflitti rilevati.

Tabella 5.3: Operazioni di `IConflictDetector`.

5.4.3 Classi

`RadarTrack.` Rappresenta una singola traccia radar, cioè una misura osservata dal sistema di sorveglianza. Include i seguenti attributi principali:

- `aircraftId : String` identificativo dell'aeromobile associato alla traccia;
- `altitude : Real` quota osservata;
- `speed : Real` velocità osservata;
- `timestamp : Integer` istante di acquisizione della traccia;
- `position : Position` con molteplicità `1..1` (ogni traccia è associata esattamente a una posizione).

`TrackingComponent.` È il componente responsabile dell'**integrazione** tra radar, registro aeromobili e conflict detection. Nel modello possiede tre riferimenti obbligatori (tutti `1..1`):

- `iradarsurveillance : IRadarSurveillance`
- `aircraftregistry : AircraftRegistry`
- `iconflictdetector : IConflictDetector`

e fornisce le operazioni:

- `void pollRadarAndUpdate()` per interrogare il radar e aggiornare lo stato degli aeromobili nel registro;
- `Conflict[*] detectConflicts()` per richiedere la rilevazione dei conflitti e restituire l'insieme dei conflitti individuati.

`Conflict.` Rappresenta un conflitto rilevato tra due aeromobili. Contiene:

- `detectedAt : Integer` timestamp (istante) di rilevazione del conflitto;
- `a1 : Aircraft` e `a2 : Aircraft` con molteplicità `1..1` (ogni conflitto coinvolge esattamente due aeromobili);
- `type : ConflictType` per classificare il tipo di conflitto.

Dal lato `Aircraft`, sono presenti gli estremi opposti delle due associazioni (`conflictA1` e `conflictA2`) con molteplicità `[*]`, indicando che un aeromobile può essere coinvolto in molteplici conflitti.

`BasicConflictDetector.` È una implementazione concreta di `IConflictDetector`. Nel modello include due parametri di configurazione:

- `minHorizontalSep : Real` separazione orizzontale minima;
- `minVerticalSep : Real` separazione verticale minima.

Il metodo `detect(...)` non è ridefinito come `ownedOperation` nella classe (viene ereditato come requisito dall'interfaccia `IConflictDetector`) ed è quindi demandato alla generazione del relativo stub nell'implementazione Java.

5.4.4 Enumerazioni

`ConflictType.` L'enumerazione `ConflictType` codifica la classificazione del conflitto con i seguenti letterali:

- `POSSIBLE_CONFLICT`
- `LOSS_OF_SEPARATION`

5.4.5 Associazioni principali

Le relazioni più rilevanti del diagramma `service` sono:

- **TrackingComponent – IRadarSurveillance**: `TrackingComponent` possiede `iradarsurveillance : IRadarSurveillance` con molteplicità `1..1`, indicando una dipendenza strutturale obbligatoria dal servizio radar.

- **TrackingComponent – AircraftRegistry**: `TrackingComponent` possiede `aircraftregistry : AircraftRegistry` con molteplicità `1..1`, in modo da poter aggiornare e consultare gli aeromobili noti al sistema.
- **TrackingComponent – IConflictDetector**: `TrackingComponent` possiede `iconflictdetecto : IConflictDetector` con molteplicità `1..1`, separando l'orchestrazione dalla logica dell'algoritmo di detection.
- **RadarTrack – Position**: ogni `RadarTrack` è associato a una `Position` con molteplicità `1..1`.
- **Conflict – Aircraft**: ogni `Conflict` è associato a due aeromobili `a1` e `a2` con molteplicità `1..1`; un singolo `Aircraft` può partecipare a molti conflitti sul lato opposto (`[*]`).

5.4.6 Gestione dei conflitti

La gestione dei conflitti emerge dal coordinamento tra `TrackingComponent` e `IConflictDetector`:

1. `pollRadarAndUpdate()` interroga il radar tramite `IRadarSurveillance.acquireTracks()`, ottenendo `RadarTrack[*]`.

2. Le informazioni delle tracce (quota, velocità, posizione, timestamp) vengono usate per aggiornare lo stato degli aeromobili nel `core.AircraftRegistry`.

3. La rilevazione dei conflitti avviene delegando a `IConflictDetector.detect(Aircraft[*])`, che produce un insieme di `Conflict[*]`.

4. Ogni `Conflict` identifica esplicitamente la coppia di aeromobili coinvolti (`a1`, `a2`), l'istante di rilevazione (`detectedAt`) e la classificazione (`type : ConflictType`).

5.4.7 Nota sugli aspetti periodici

Nel modello, `TrackingComponent` è annotato con lo stereotipo *Periodic* (con `periodMs = 1000` per `TrackingComponent`). Questa informazione consente di rappresentare esplicitamente nel modello la natura temporale di alcune attività e, se previsto dal generatore, può influenzare il codice generato (ad esempio introducendo un attributo di periodo e la relativa inizializzazione).

5.5 Class Diagram: comms

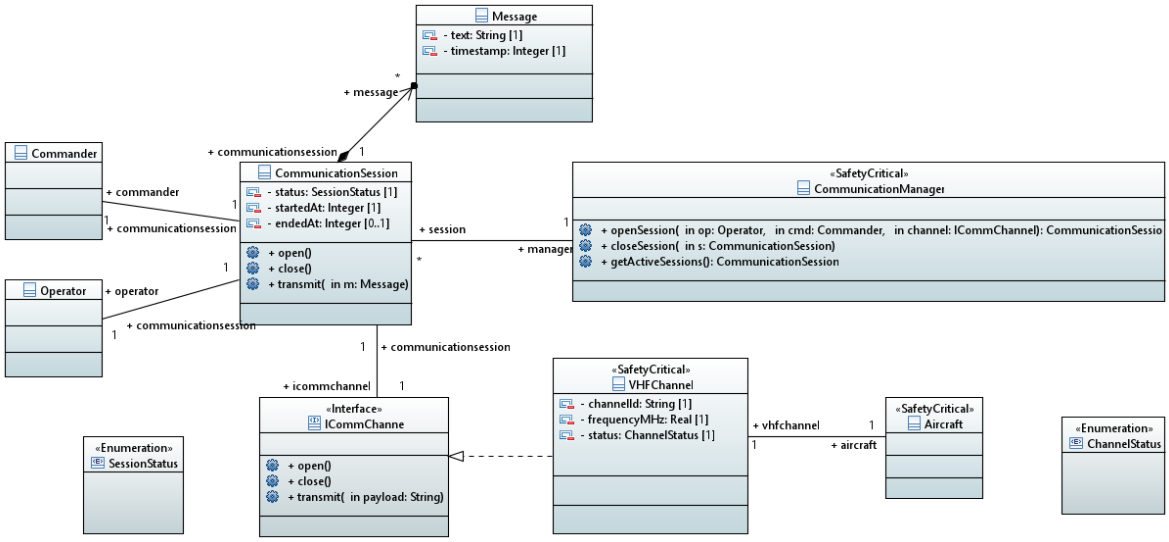

Figura 5.5: Cm_ClassDiagram

5.5.1 Panoramica

Il package `comms` modella il sottosistema di **comunicazione** del sistema ATC, con particolare attenzione a: (i) gestione di sessioni di comunicazione tra attori (operatore e comandante), (ii) astrazione dei canali di comunicazione e implementazione VHF, (iii) scambio di messaggi e tracciamento dello stato (canale/sessione). La modellazione separa in modo esplicito **interfacce** e **implementazioni**, rendendo il sistema estendibile verso ulteriori tipi di canali oltre al VHF.

5.5.2 Interfacce

`ICommChannel.` L'interfaccia `ICommChannel` rappresenta un canale di comunicazione astratto. Espone le seguenti operazioni:

Operazione	Descrizione
`void open()`	Apre/attiva il canale di comunicazione.
`void close()`	Chiude/disattiva il canale di comunicazione.
`void transmit(String payload)`	Trasmette un payload testuale sul canale.

Tabella 5.4: Operazioni di `ICommChannel`.

5.5.3 Classi

`VHFChannel.` `VHFChannel` è l'implementazione concreta del canale VHF e **realizza** (`implements`) l'interfaccia `ICommChannel`. Nel modello include i seguenti attributi:

- `channelId : String`: identificativo del canale;
- `frequencyMHz : Real`: frequenza operativa del canale;
- `status : ChannelStatus`: stato corrente del canale.

Nel diagramma `VHFChannel` non definisce operazioni come `ownedOperation`: le operazioni `open/close/transmit` sono **richieste** dall'interfaccia `ICommChannel` e quindi vengono implementate a livello di generazione Java tramite metodi `@Override` (stub).

`CommunicationSession.` `CommunicationSession` modella una sessione di comunicazione tra le parti coinvolte e gestisce lo scambio di messaggi nel tempo. Attributi:

- `status : SessionStatus`: stato della sessione (aperta/chiusa);
- `startedAt : Integer`: timestamp di apertura;
- `endedAt : Integer` con molteplicità `0..1`: timestamp di chiusura (presente solo quando la sessione termina);
- `message : Message[*]` con molteplicità `0..*`: collezione dei messaggi scambiati nella sessione.

Operazioni:

- `void open()` e `void close()` per la gestione del ciclo di vita della sessione;
- `void transmit(Message m)` per trasmettere un messaggio a livello di sessione.

`CommunicationManager.` `CommunicationManager` gestisce la creazione e la chiusura delle sessioni e fornisce l'accesso all'insieme delle sessioni attive. Nel modello espone:

Operazione	Descrizione
`CommunicationSession openSession(Operator op, Commander cmd, ICommChannel channel)`	Apre una sessione tra operatore e comandante, associandola ad un canale astratto.
`void closeSession(CommunicationSession s)`	Chiude una sessione specificata.
`CommunicationSession[*] getActiveSessions()`	Restituisce l'insieme delle sessioni attive (molteplicità di ritorno `0..*`).

Tabella 5.5: Operazioni di `CommunicationManager`.

`Message`. `Message` rappresenta l'unità informativa scambiata durante una sessione. Attributi:

- `text : String`: contenuto testuale del messaggio;
- `timestamp : Integer`: istante di creazione/trasmissione.

5.5.4 Enumerazioni

Il modello definisce due enumerazioni per rendere esplicito lo stato del sottosistema:

- `ChannelStatus` con letterali: `FREE`, `BUSY`, `OFFLINE`.
- `SessionStatus` con letterali: `OPEN`, `CLOSED`.

5.5.5 Associazioni principali

Nel diagramma `comms` sono presenti le seguenti relazioni strutturali (con molteplicità come definita nel modello):

- `CommunicationManager` – `CommunicationSession`: un `CommunicationManager` è associato a `session : CommunicationSession[*]` con molteplicità `0..*`; ogni `CommunicationSession` è associata a un `manager : CommunicationManager` con molteplicità `1..1`.
- `CommunicationSession` – `Operator`: associazione `1..1`, che collega ogni sessione a un operatore (`domain.Operator`).
- `CommunicationSession` – `Commander`: associazione `1..1`, che collega ogni sessione a un comandante (`domain.Commander`).

- `CommunicationSession` – `ICommChannel`: associazione `1..1` verso un canale astratto (`ICommChannel`), consentendo l'uso di implementazioni differenti (es. `VHFChannel`).
- **Composizione** `CommunicationSession` – `Message`: la sessione **compone** `message : Message[*]` con molteplicità `0..*`, indicando che i messaggi appartengono alla sessione e ne seguono il ciclo di vita.
- `domain.Aircraft` – `VHFChannel`: associazione `1..1` tra un aeromobile e un canale VHF (modellazione della disponibilità/assegnazione del canale all'aeromobile).

5.5.6 Implicazioni sulla generazione Java

Il diagramma `comms` introduce diversi casi che la trasformazione Acceleo deve gestire correttamente:

- **Interfacce e realizzazioni**: `VHFChannel` deve includere `implements ICommChannel` e l'implementazione dei metodi `open/close/transmit` (tipicamente tramite stub e annotazione `@Override`).
- **Molteplicità** `0..*`: la proprietà `message : Message[*]` e il ritorno di `getActiveSessions(): CommunicationSession[*]` devono essere tradotti in Java come collezioni (ad esempio `List<Message>` e `List<CommunicationSession>`), includendo gli import necessari.
- **Enum**: `ChannelStatus` e `SessionStatus` devono essere rappresentati nel codice (come `enum` dedicati o come tipi referenziati coerentemente, a seconda delle funzionalità supportate dal generatore).

Capitolo 6

Trasformazione Acceleo

6.1 Introduzione alla generazione Acceleo

L'obiettivo della seconda parte della task è realizzare una trasformazione *model-to-text* (M2T) con **Acceleo** in grado di generare automaticamente una prima implementazione Java a partire dal modello UML del sistema ATC. Il risultato atteso non è un'applicazione completa, ma uno **skeleton coerente** con il modello: package, classi/interfacce/enum, attributi (inclusi quelli derivanti da associazioni navigabili), firme dei metodi e gestione di ereditarietà e realizzazioni di interfacce.
La trasformazione è stata progettata con i seguenti principi:

- **Coerenza con UML**: nomi, visibilità, tipi e molteplicità sono generati a partire dagli elementi UML.
- **Copertura dei tipi principali**: vengono generati `Class`, `Interface` ed `Enumeration`.
- **Relazioni strutturali**: l'ereditarietà è tradotta con `extends`, le interfacce con `implements`; le associazioni navigabili diventano campi (riferimento singolo o collezione).
- **Preservazione del codice manuale**: il codice che lo sviluppatore inserisce nei punti previsti non viene sovrascritto alle rigenerazioni (aree protette).

Nel seguito si descrive nel dettaglio la struttura di `generate.mtl`, le query di supporto per il mapping dei tipi, la gestione degli stereotipi (in particolare `Periodic`) e, soprattutto, la gestione delle associazioni e dell'ereditarietà così come realizzata nel progetto.

6.2 Struttura del modulo `generate.mtl`

Questa sezione descrive l'organizzazione del modulo Acceleo `generate.mtl`, evidenziando le macro-sezioni utilizzate per ottenere una generazione Java completa e coerente

con il modello UML.

6.2.1 Intestazione del modulo e template di ingresso

Il modulo importa i metamodel `Types` e `UML`. Il punto di ingresso della trasformazione è un template che riceve in input un `Package` UML e itera su tutti i tipi contenuti nei package del modello, generando il file Java appropriato in base al tipo UML (classe/interfaccia/enum).

```
[comment encoding = UTF-8 /]
[module generate('http://www.eclipse.org/uml2/5.0.0/Types',
                 'http://www.eclipse.org/uml2/5.0.0/UML')]

[template public GenerateJava(aPackage : Package)]
[for (t : Type | aPackage.eAllContents(Package).ownedType)]
  [if (t.oclIsTypeOf(Class))]
    [GenerateJavaClass(t.oclAsType(Class))/]
  [/if]
  [if (t.oclIsTypeOf(Interface))]
    [GenerateJavaInterface(t.oclAsType(Interface))/]
  [/if]
  [if (t.oclIsTypeOf(Enumeration))]
    [GenerateJavaEnum(t.oclAsType(Enumeration))/]
  [/if]
[/for]
[/template]
```

Listing 6.1: Header e template di ingresso con dispatch sui tipi

6.2.2 Guard di template, cosa generare

Durante l'evoluzione del progetto è utile ricordare che Acceleo consente di filtrare l'applicazione di un template tramite una guard (predicato) posta nella firma del template, dopo i parametri.
Ad esempio, una versione iniziale che genera solo classi marcate con lo stereotipo `Periodic` può essere espressa così:

```
[template public GenerateJavaClass(aClass : Class)
 ? (aClass.getAppliedStereotypes()->exists(s |
    s.name='Periodic'))]
[/template]
```

Listing 6.2: Esempio di template con guard: genera solo classi Periodic

Nel progetto attuale la guard è stata rimossa per generare tutte le classi del modello.

6.2.3 Direttiva `package` e area protetta degli import

Ogni file generato inizia con la direttiva `package` coerente con il package UML di appartenenza e contiene un'area protetta dedicata agli import manuali. L'area protetta consente di inserire import aggiuntivi senza perderli in caso di rigenerazione.

```
[file (aClass.name.concat('.java'), false, 'UTF-8')]
package [aClass.namespace.name/]
//[protected ('custom imports - preserved on regeneration')]
import ... // add packages to be imported
//[/protected]
```

Listing 6.3: Package e area protetta per import personalizzati

6.2.4 Firma della classe: `extends` e `implements`

La firma della classe viene generata includendo:

- **extends** se è presente una generalizzazione (ereditarietà singola in Java);
- **implements** se la classe realizza una o più interfacce.

```
public class [aClass.name.toUpperFirst()/]
[if (aClass.generalization->size() > 0)]
  extends [aClass.generalization->any(true).general.name/]
[/if]
[if (aClass.getAllImplementedInterfaces()->size() > 0)]
  implements [for (i : Interface |
     aClass.getAllImplementedInterfaces())
              separator(', ')][i.name/][/for]
[/if]
{
```

Listing 6.4: Generazione della firma della classe

6.2.5 Generazione attributi: proprietà e associazioni navigabili

Gli attributi vengono generati iterando sulle `Property` restituite da `aClass.attribute`. In questo modo, oltre agli attributi "semplici" (primitive type o classifier), vengono generati anche i ruoli di associazione navigabili modellati come proprietà della classe.

```
[for (p : Property | aClass.attribute)]
  [p.visibility.toString()/] [p.jType()/] [p.name/];
[/for]
```

Listing 6.5: Loop di generazione dei campi

6.2.6 Costruttori, getter/setter

Se una classe possiede almeno una proprietà, viene generato un costruttore con parametri pari a tutte le proprietà; quindi vengono generati i relativi getter e setter. Questa scelta garantisce uno skeleton immediatamente utilizzabile per istanziare e manipolare gli oggetti senza sovraccaricare di operazioni i file UML.

6.2.7 Operazioni: owned e metodi richiesti da interfacce/astratti

Il generatore distingue tra:

- **operazioni owned** (`aClass.ownedOperation`), per cui vengono generate le firme e il corpo con area protetta;
- **operazioni da implementare** provenienti da interfacce realizzate o da metodi astratti ereditati, per cui vengono generati stub con `@Override`.

6.2.8 Aree protette nei metodi

Ogni metodo generato contiene una sezione protetta, che in Java appare come: `Start of user code ...` `End of user code`. Il contenuto all'interno è preservato tra rigenerazioni, consentendo di completare i metodi a mano senza perdere modifiche.

6.3 Gestione degli stereotipi del profilo ATC

Nel progetto viene applicato il profilo `ATC_Profile`, che introduce tre stereotipi:

- `Periodic` (su `Class`): aggiunge la proprietà `periodMs : Integer` per modellare componenti a comportamento periodico;
- `SafetyCritical` (su `Class`): aggiunge la proprietà `criticality : CriticalityLevel` (HIGH/MEDIUM/LOW) per evidenziare componenti critici dal punto di vista della sicurezza;
- `Subsystem` (su `Package`): permette di categorizzare package come sottosistemi architetturali.

Nel modello ATC, `Periodic` è effettivamente applicato a `service.TrackingComponent` (con `periodMs=1000`). Lo stereotipo `SafetyCritical` è applicato a più classi: ad esempio `ATCSystem` e `AircraftRegistry` con criticità `MEDIUM`, e `Aircraft`, `VHFChannel` e `RadarTrack` con criticità `LOW`; inoltre risulta applicato anche a `TrackingComponent`, `BasicConflictDetector` e `CommunicationManager`.
Dal punto di vista della generazione, il template Acceleo sfrutta `Periodic` per introdurre nel Java generato un attributo dedicato (es. `periodMs`) e la relativa inizializzazione nei costruttori. Con `SafetyCritical` aggiungiamo invece l'attributo `criticality` mentre `Subsystem` viene mantenuto solo come metadato di analisi/documentazione.

6.4 Gestione delle associazioni: casi e decisioni

La gestione delle associazioni è fondamentale per mantenere la coerenza tra modello UML e codice Java. Nel progetto si è adottata una scelta semplice e robusta: un'associazione viene generata come campo Java solo se l'estremità è modellata come `Property` della classe (cioè come ruolo navigabile/owned). In questo modo, attributi semplici e riferimenti verso altre classi vengono trattati in maniera uniforme.

6.4.1 Molteplicità e tipi: riferimento singolo vs collezione

La differenza principale tra i casi di associazione è la molteplicità. Per evitare duplicazione di logica, il mapping del tipo Java è centralizzato in una query `jType`:

```
[query public jType(p : Property) : String =
  if p.type = null then
    'Object'
  else if p.type.name = 'Real' then
    'Double'
  else
    if (p.upper = -1 or p.upper > 1) then
      'java.util.List<'.concat(p.getPropertyType()).concat('>')
    else
      p.getPropertyType()
    endif
  endif
/]
```

Listing 6.6: Query `jType`: mapping tipo + molteplicità

Interpretazione.

- **1..1** e **0..1** → **riferimento singolo** (`Type`).
- **0..*** e **1..*** → **collezione** (`List<Type>`).

6.4.2 Caso 1: associazione 1–1 (riferimento obbligatorio)

Se la molteplicità è `1..1`, il generatore crea un campo singolo del tipo associato. **Esempio:** `TrackingComponent.iradarsurveillance : IRadarSurveillance[1..1]` diventa un riferimento singolo:

```
public IRadarSurveillance iradarsurveillance;
```

Listing 6.7: Esempio (1..1) generato come riferimento singolo

6.4.3 Caso 2: associazione 0–1 (riferimento opzionale)

Se la molteplicità è `0..1`, il campo resta singolo ma è **opzionale** (in Java tipicamente rappresentato come `null` quando assente). **Esempio:** `ATCSystem.communicationmanager : CommunicationManager[0..1]`:

```
public CommunicationManager communicationmanager; // puo' essere
    null
```

Listing 6.8: Esempio (0..1) generato come riferimento singolo opzionale

6.4.4 Caso 3: associazione 0–* / 1–* (collezione)

Se la molteplicità è multivalore (`0..*` o `1..*`), il generatore usa una collezione `List<T>`. **Esempio:** `CommunicationSession.message : Message[0..*]`:

```
public java.util.List<Message> message;
```

Listing 6.9: Esempio (0..*) generato come collezione

6.4.5 Composizione vs associazione

Nel modello alcune relazioni sono indicate come **composizioni**. A livello di **codice Java generato**, la composizione viene comunque rappresentata come un campo (singolo o collezione) come negli altri casi: la differenza è semantica e viene mantenuta nel modello.

6.4.6 Associazioni presenti nel modello ma non generate come campi

Se un'associazione è definita nel modello ma le sue estremità non sono modellate come `Property` owned/navigabile della classe, allora non compare nel loop `aClass.attribute` e quindi non viene generata come campo. In questi casi, il legame può comunque emergere:

- come **parametro** di un metodo;
- come **tipo di ritorno** (anche collezione) di un metodo.

Esempio: il legame tra manager e sessioni può essere espresso tramite un metodo che ritorna `CommunicationSession[*]` e viene tradotto in `List<CommunicationSession>` (tramite la query di ritorno).

6.4.7 Bidirezionalità: scelta progettuale

Nel progetto la generazione risulta **unidirezionale** rispetto alle proprietà effettivamente navigabili/owned: non vengono creati automaticamente campi "speculari" per rendere le associazioni bidirezionali, evitando duplicazione di stato e necessità di sincronizzazione tra estremità.

6.5 Gestione dell'ereditarietà: casi e decisioni

La trasformazione gestisce l'ereditarietà su due livelli:

- **generalizzazione tra classi** (`extends`);
- **realizzazione di interfacce** (`implements`) e generazione degli stub dei metodi richiesti.

6.5.1 Ereditarietà tra classi: `extends`

Nel modello ATC sono presenti generalizzazioni nel package `domain`. In particolare:

- `Operator extends Person`
- `Commander extends Person`

Il template genera `extends` quando la classe ha almeno una generalizzazione. Poiché Java supporta ereditarietà singola, il generatore considera la prima generalizzazione disponibile:

```
public class [aClass.name.toUpperFirst()/]
[if (aClass.generalization->size() > 0)]
  extends [aClass.generalization->any(true).general.name/]
[/if]
```

Listing 6.10: Generazione `extends` da generalizzazione UML

Nel codice generato questo produce, ad esempio:

```
public class Operator extends Person {
    ...
}
```

Listing 6.11: Esempio: Operator estende Person

6.5.2 Interfacce: `implements` e generazione degli stub

Il generatore aggiunge `implements` quando una classe realizza una o più interfacce. Nel modello ATC sono presenti casi significativi, ad esempio:

- `BasicConflictDetector implements IConflictDetector`
- `VHFChannel implements ICommChannel`

```
[if (aClass.getAllImplementedInterfaces()->size() > 0)]
  implements [for (i : Interface |
     aClass.getAllImplementedInterfaces())
              separator(', ')][i.name/][/for]
[/if]
```

Listing 6.12: Generazione `implements` per interfacce UML

6.5.3 Metodi da implementare

Per garantire che il codice compilabile (o almeno strutturalmente coerente) includa i metodi richiesti, il template calcola l'insieme delle operazioni da implementare:

- tutte le operazioni delle interfacce realizzate;
- tutte le operazioni astratte ereditate dalle superclassi;
- escludendo quelle già definite come owned operation nella classe corrente.

```
[let interfaceOps : Sequence(Operation) =
   aClass.getAllImplementedInterfaces().ownedOperation
 /]
[let abstractInheritedOps : Sequence(Operation) =
   aClass.allParents().ownedOperation->select(o | o.isAbstract)
 /]
[let operationsToImplement : Sequence(Operation) =
   interfaceOps->union(abstractInheritedOps)->select(op |
      not aClass.ownedOperation->exists(owned |
          owned.name = op.name and owned.ownedParameter->size()
             = op.ownedParameter->size()
      )
   )
 /]
```

Listing 6.13: Selezione delle operazioni da implementare

Per ciascuna operazione selezionata, il generatore produce uno stub con `@Override` e un corpo in area protetta, così da preservare un'eventuale implementazione del body della funzione.

Capitolo 7

Parsing

7.1 Introduzione

Il punto 3 della task richiede di realizzare una trasformazione analoga a quella Acceleo, ma usando un linguaggio general-purpose (Java) e una libreria di parsing XML (nel progetto è stato scelto **SAX**). L'idea è replicare la pipeline *model-to-text* in modo esplicito:

1. lettura del file `.uml` (XMI generato da Papyrus) tramite parser SAX;
2. costruzione di un modello intermedio in memoria (`UmlModel`, `UmlType`);
3. generazione dei file Java a partire dal modello intermedio (`JavaCodeGenerator`).

Questo approccio rende trasparenti sia le scelte di parsing del metamodel UML (elementi XMI letti e interpretati), sia le regole di mapping verso Java (package, tipi, molteplicità, import, `extends`/`implements`, operazioni).

7.2 Entry point: `Main.java`

Il punto di ingresso dell'esecuzione è la classe `ATC_gen.Main`. Essa ha il ruolo di orchestrare l'intera pipeline del punto 3: seleziona l'input, invoca il parser sul file UML/XMI e, a partire dal modello intermedio ottenuto, avvia la generazione del codice Java nella cartella di output.

```
package ATC_gen;

import ATC_genCodegen.JavaCodeGenerator;
import ATC_genModel.UmlModel;
import ATC_genParser.PapyrusUmlSaxParser;
import java.nio.file.Path;
```

```java

public class Main {

    //classe main.java che inizia ed esegue la generazione

    public static void main(String[] args) throws Exception {

        String inputPath = null;            // null => usa resources/model/ATC_Model.uml
        String outputDir = "generated-src"; // default

        if (args.length == 1) {
            if (args[0].toLowerCase().endsWith(".uml")) {
                inputPath = args[0];
            } else {
                outputDir = args[0];
            }
        } else if (args.length >= 2) {
            inputPath = args[0];
            outputDir = args[1];
        }

        UmlModel model = PapyrusUmlSaxParser.parse(inputPath);

        System.out.println("=== TYPES FOUND ===");
        for (int i = 0; i < model.getTypes().size(); i++) {
            System.out.println(" - " + model.getTypes().get(i).kind + "  " + model.getTypes().get(i).qualifiedName());
        }

        JavaCodeGenerator.generate(model, Path.of(outputDir));

        System.out.println("=== DONE ===");
        System.out.println("Output folder: " + Path.of(outputDir).toAbsolutePath());
    }
}
```

Listing 7.1: Main del punto 3: avvio parsing + generazione

Come funziona in pratica. Il metodo `main` esegue tre passi principali:

1. **Gestione parametri e scelta input/output.**

   - `java Main`: `inputPath` resta `null` e il parser utilizza il modello di default nelle risorse (`src/main/resources/model/ATC_Model.uml`); l'output va in `generated-src`.
   - `java Main path/modello.uml`: se l'unico argomento termina con `.uml`, viene interpretato come percorso del modello; l'output resta `generated-src`.
   - `java Main outFolder`: se l'unico argomento non termina con `.uml`, viene interpretato come cartella di output; l'input resta quello di default.
   - `java Main path/modello.uml outFolder`: con due argomenti si specificano esplicitamente sia input sia output.

   Questa logica rende l'esecuzione flessibile senza introdurre parsing complesso degli argomenti.

2. **Parsing del modello UML.** La chiamata `PapyrusUmlSaxParser.parse(inputPath)` è il punto in cui il file `.uml` (XMI generato da Papyrus) viene letto e trasformato in un `UmlModel` in memoria. Il `Main` non entra nei dettagli del parsing: si limita a richiedere il modello intermedio, mantenendo una chiara separazione di responsabilità.

3. **Generazione del codice e diagnostica.** Prima della generazione, il `Main` stampa a console un riepilogo dei tipi individuati nel modello (categoria e nome qualificato). Questa stampa serve come controllo rapido: consente di verificare che il parser abbia riconosciuto correttamente classi/interfacce/enum e la struttura dei package. Infine, `JavaCodeGenerator.generate(...)` produce i file Java nella cartella scelta e viene stampato il percorso assoluto dell'output per facilitare il recupero dei file generati.

Comportamento.

- **Input**: se non viene passato alcun parametro, il parser usa il modello incluso nelle risorse (`src/main/resources/model/ATC_Model.uml`). Se viene passato un percorso `.uml`, quel file viene usato come input.
- **Output**: di default la generazione produce i file in `generated-src`. Se viene passato un percorso non `.uml`, viene interpretato come cartella di output.
- **Diagnostica**: stampa a console i tipi trovati (kind + qualified name) prima di generare.

7.3 Componenti principali

7.3.1 PapyrusUmlSaxParser

PapyrusUmlSaxParser è il componente responsabile del parsing SAX del file UML/XMI. Il parser è namespace-aware e interpreta gli elementi principali generati da Papyrus/UML2, tra cui:

- packagedElement per riconoscere Package, Class, Interface ed Enumeration;
- ownedAttribute per estrarre attributi e ruoli (con tipo e visibilità);
- ownedOperation e ownedParameter per ricostruire firme dei metodi (incluso return);
- lowerValue/upperValue per catturare la molteplicità (incluso *);
- generalization e interfaceRealization per extends e implements;
- ownedLiteral per i letterali delle enumerazioni.

Per gestire la nidificazione dei package e la corretta associazione di attributi/operazioni al tipo corrente, il parser mantiene strutture a stack (package correnti, tipo corrente, operazione corrente, parametro corrente).

7.3.2 UmlModel

UmlModel rappresenta il contenitore del modello intermedio costruito dal parser. Mantiene:

- una lista ordinata dei tipi trovati nel file (List<UmlType>);
- una mappa id → UmlType per risolvere riferimenti tra tipi (necessaria per extends, implements, tipi di attributi e parametri).

In pratica, UmlModel funge da "indice" per collegare tra loro i vari elementi letti dall'XMI.

7.3.3 UmlType

UmlType è la rappresentazione intermedia di un classifier UML e supporta tre categorie (Kind): CLASS, INTERFACE, ENUM. Per ogni tipo vengono memorizzati:

- identificativo XMI (id), nome e package qualificato;
- superclasse (superTypeId) e lista di interfacce realizzate (interfaceIds);
- lista di attributi (UmlAttribute) con tipo e molteplicità;
- lista di operazioni (UmlOperation) con parametri (UmlParameter) e tipo di ritorno;
- eventuali letterali (nel caso ENUM).

Sono inoltre presenti helper per interpretare la molteplicità e riconoscere i casi multivalore (upper `*`, `-1` o numerico > 1).

7.3.4 `JavaCodeGenerator`

`JavaCodeGenerator` implementa la fase **model-to-text** vera e propria. A partire dall'`UmlModel`:

- crea le cartelle di package in output e genera un file `.java` per ogni tipo;
- risolve i tipi (primitive da `href #String/Integer/Real` e tipi del modello tramite `xmi:id`);
- traduce la molteplicità in `List<T>` quando necessario e aggiunge automaticamente gli import (es. `java.util.List`);
- genera firme `extends`/`implements` coerenti con generalizzazioni e realizzazioni;
- per le classi genera campi, costruttori, getter/setter e stub delle operazioni (incluse quelle richieste da interfacce o ereditarietà astratta);
- nei metodi inserisce marker in stile "user code" per indicare le sezioni destinate a essere completate manualmente.

7.4 Risultato dell'approccio e funzionamento complessivo

In conclusione, questo approccio produce una generazione Java end-to-end a partire dal file UML/XMI di Papyrus, senza passare da un template M2T dichiarativo (come Acceleo), ma implementando esplicitamente le due fasi fondamentali:

- **Parsing (SAX)**: il file `.uml` viene letto come XML e convertito in un modello intermedio (`UmlModel`) che contiene i tipi (`UmlType`), gli attributi, le operazioni, le generalizzazioni e le molteplicità.
- **Generazione (Java)**: il `JavaCodeGenerator` percorre il modello intermedio e scrive i corrispondenti file `.java` nei package di output, generando firme, campi e metodi coerenti con i dati estratti.

Il motivo per cui il processo funziona è che Papyrus serializza il modello UML in formato XMI con una struttura regolare: il parser riconosce gli elementi rilevanti (tipi, attributi, operazioni, relazioni) e li ricostruisce in una rappresentazione interna più semplice da usare per la stampa dei sorgenti. In questo modo, il punto 3 realizza una trasformazione M2T *programmata*, in cui le regole di mapping e la produzione dei file vengono controllate direttamente in Java.

Capitolo 8

Confronto tra i due approcci

8.1 Introduzione

Per soddisfare le richieste della task sono stati sperimentati due approcci distinti di trasformazione *Model-to-Text* per generare codice Java a partire dal modello UML:

- **Approccio dichiarativo con Acceleo**: la trasformazione è espressa tramite template M2T (`.mtl`) che navigano il metamodel UML (via EMF/UML2) e producono direttamente i file `.java`.
- **Approccio programmativo con parsing SAX in Java**: il file `.uml` (serializzato in XMI) viene letto come XML; si costruisce un modello intermedio (`UmlModel`/`UmlType`) e infine si genera Java tramite un generatore dedicato.

L'obiettivo di questa sezione è discutere in modo critico **pro e contro** di entrambi i metodi, evidenziando le implicazioni su sviluppo, manutenzione e robustezza.

8.2 Approccio Acceleo: punti di forza e criticità

8.2.1 Pro

- **Semplicità e concisione**: Acceleo è progettato specificamente per M2T. Una grande parte della logica di generazione (iterazione su classi, produzione di file, gestione di output) è espressa in poche righe, con un rapporto *funzionalità/LOC* tipicamente molto alto rispetto a un'implementazione equivalente in Java.
- **Allineamento naturale con UML**: il template lavora direttamente su concetti UML (Class, Property, Operation, Generalization, ecc.). Questo riduce il rischio di "interpretare male" la struttura XMI e rende le regole di mapping più leggibili e vicine al dominio.

- **Evoluzione rapida**: modificare la forma del codice generato (aggiungere una sezione, cambiare una firma, gestire un caso di molteplicità) richiede spesso interventi localizzati nel template, senza introdurre architetture o parsing complessi.
- **Rigenerazione controllata**: le *protected regions* sono un vantaggio pratico importante: consentono di rigenerare mantenendo le porzioni implementate manualmente, facilitando un workflow iterativo modello→codice.
- **Manutenibilità del mapping**: l'uso di **query** per il mapping (es. tipi e collezioni) centralizza le regole: la trasformazione resta più pulita e coerente.

8.2.2 Contro

- **Debugging meno immediato**: rispetto a Java, il debugging di template può risultare meno intuitivo (specie per errori su OCL/iterazioni), e richiede familiarità con l'ambiente Eclipse/Acceleo.
- **Logiche molto complesse**: quando la generazione richiede trasformazioni sofisticate (ottimizzazioni, analisi globali, scheduling, ecc.), un linguaggio general-purpose può offrire maggiore espressività e tooling.
  - **Tempi di generazione**: l'esecuzione dei template Acceleo e l'accesso al modello via EMF/UML2 introducono un overhead; di conseguenza la generazione dei file Java può risultare più lenta rispetto a un generatore implementato direttamente in Java.
  - **Tempi di esecuzione della pipeline**: soprattutto su modelli grandi, l'interpretazione dei template e la navigazione del metamodel possono aumentare i tempi complessivi della trasformazione.
- **Dipendenze dall'ecosistema EMF/UML2**: l'approccio è molto efficace nel contesto Eclipse, ma meno portabile in pipeline esterne se non si integra lo stack EMF/Acceleo.

8.3 Approccio Java + SAX: punti di forza e criticità

8.3.1 Pro

- **Massimo controllo**: implementando parsing e generazione in Java si controllano in modo fine: gestione errori, log, performance, validazioni, policy di naming, formattazione, ecc.
- **Integrazione e test**: un progetto Java è semplice da integrare in CI/CD, da coprire con test automatici e da mantenere con strumenti standard (JUnit, static analysis, build tool).

- **Velocità di esecuzione**: un approccio in Java (parsing SAX + generazione) tende ad essere più veloce, perché evita l'overhead dell'engine M2T e lavora in modo più diretto su strutture dati ottimizzabili.
- **Estendibilità generale**: se in futuro la sorgente non fosse più solo UML (o fosse UML con varianti/tool diversi), un approccio programmativo consente più facilmente di gestire casi speciali, normalizzazioni o importazioni eterogenee.

8.3.2 Contro

- **Molto più codice e complessità**: bisogna implementare sia il *parsing* XMI (riconoscere tag, attributi, namespace, riferimenti, molteplicità) sia la costruzione di un modello intermedio e poi la generazione. Questo porta inevitabilmente a più righe di codice, più classi e maggiore sforzo di manutenzione.
- **Fragilità rispetto al formato XMI**: il parsing SAX è legato alla serializzazione concreta prodotta da Papyrus/UML2. Cambi di versione, differenze tra tool o variazioni di esportazione possono richiedere adattamenti del parser.
- **Rischio di mismatch semantici**: lavorando su XML ci si allontana dai concetti UML "puri"; interpretare correttamente generalizzazioni, interfacce, molteplicità e stereotipi richiede attenzione e può introdurre bug sottili.

Capitolo 9

Conclusioni

In questo progetto è stata realizzata una pipeline M2T completa per il caso di studio ATC, sperimentando due modalità di generazione Java a partire da UML. Il generatore **Acceleo** ha mostrato come un approccio dichiarativo consenta di produrre rapidamente codice coerente con il modello, gestendo in modo naturale ereditarietà, associazioni, metodi e rigenerazione controllata tramite *protected regions*. L'approccio **Java+SAX** ha invece evidenziato la flessibilità di una soluzione programmativa, a fronte di un maggiore carico implementativo e di una dipendenza più marcata dal formato XMI.
Nel complesso, per una generazione standard di implementazioni a partire da UML, l'approccio basato su **Acceleo** risulta più semplice, compatto e manutenibile; la soluzione Java rimane una valida alternativa quando si richiedono estensioni avanzate o integrazioni fuori dall'ecosistema MDE. Questo confronto permette di motivare in modo chiaro la scelta degli strumenti e di inquadrare correttamente i compromessi tra semplicità e controllo.